Chapter 12. Matrix Extensions

This chapter is a placeholder for forthcoming matrix extensions.

12.1. Zvt Family of Vector Matrix Extensions, Version 0.4

12.1.1. Overview

The Zvt family of extensions supports high-throughput matrix-multiplication computation. These extensions add new architectural state to each hart in the form of square matrix tiles. They provide instructions that perform tall-skinny by short-wide matrix multiplications, i.e. outer products or wide outer products. These instructions source their operands from vector registers and accumulate their results into the new square matrix tiles. The inputs' data width is specified by SEW, while the accumulators' data width is given by TEW=SEW*TWIDEN, where TWIDEN can be 1, 2, or 4, depending on the instruction.

The new computational instructions perform the operation C[M,N] += A[K,M]^T*B[K,N], where C is stored in the tile state. Each of the input matrices A and B is held in a set of vector registers, where each row of the matrix is held in a separate vector register group and the number of rows is given by K. The inner dimension K can be a maximum (KMAX) of 1, 2, or 4, depending on the data type of A and B.

In addition, the Zvt extensions provide instructions to move rows and columns of the matrix arrays, either to and from the vector registers or to and from memory.

The Zvtbase extension provides the new architectural state and the new configuration, move, load, store, and context-manipulation instructions. In the base, the move, load, and store instructions support a tile EEW (TEW) of 32; other TEWs are reserved for these instructions.



Not mandating support for all TEWs reduces hardware complexity. Mandating all Zvt implementations support TEW=32 allows the context-switch routines to be the same for all implementations. TEW=32 was chosen because it is currently the most popular accumulator width, and other aspects of the Zvt extensions are optimized accordingly.

Moves, loads, and stores for other TEW values are provided by additional extensions.

Extension	Dependencies	Description
Zvtbase	Zve32x	Tile state; TEW=32 moves, loads, and stores
Zvt8e	Zvtbase	TEW=8 moves, loads, and stores
Zvt16e	Zvtbase	TEW=16 moves, loads, and stores
Zvt64e	Zvtbase_Zve64x	TEW=64 moves, loads, and stores

The Zvt extensions build upon the vector extensions. The vector registers serve as a data source for matrix operands, and the existing vector instructions are used for pre- and post-processing of matrices. Hence, the Zvtbase extension requires the Zve32x extension.

The Zvt extensions are compatible with the RV32E, RV32I, RV64E, and RV64I base ISAs, but it is expected that RV64I will be by far the most common base-ISA choice.

The following table lists the computational extensions in the Zvt family.

Extension	Dependencies	Multiplicand Types	Accumulator Type
Zvti8i32mm	Zvtbase_Zvt8e	[U]Int8	Int32
Zvtofp8fmm	Zvtbase_Zvt8e_Zve32f	OCP MX FP8	FP32
Zvtfp16fmm	Zvtbase_Zvt16e_Zve32f	FP16	FP32
Zvtbf16fmm	Zvtbase_Zvt16e_Zve32f	BF16	FP32
Zvtfmm	Zvtbase_Zve32f	FP32	FP32
Zvtdmm	Zvtbase_Zvt64e_Zve64d	FP64	FP64

Each computational extension requires the moves, loads, and stores of both the multiplicand width and accumulator width be supported.



Requiring transfers of multiplicand width be supported facilitates efficient transpose of matrices of those datatypes using the tile storage.

12.1.1.1. Matrix Tile State

The new matrix state is organized into *matrix tiles*, where a matrix tile is a separately addressable two-dimensional square matrix. The fixed matrix state is viewed as a different number of tiles, depending on TEW.



For brevity, we use the term tile as shorthand for matrix tile. We considered instead naming these tiles accumulator tiles, but chose not to do so because they are used for other purposes, including matrix transposition.

The ISA is optimized for the case where TEW=32, as this is the most common accumulator size. When TEW=32, the tiles have dimensions TExTE elements, where TE is a parameter of the implementation. TE is constrained to be a power of 2, such that VLEN/4 >= TE >= 4. The upper bound is set by the requirement that a tile row or column must fit within a single vector register group (VLEN*8 bits). Implementations may choose less than the maximum TE to reduce implementation cost. The configuration instructions, described below, support writing of TE-agnostic code.

The ISA fixes the number of TEW=32 tiles to four (`mt0`, `mt4`, `mt8`, `mt12`) to support all cases at reasonable performance. The matrix state needs to hold at least one accumulator as well as transposes of one or both inputs to support efficient execution of all matrix operations. Also, providing more separately addressable tiles allows more flexible usage of tile accumulators enabling optimizations such as register blocking of the operands, reducing memory traffic.

```
Tile EEW  NTiles Tile Dimensions Tile Specifiers
  8         16         TExTE       mt0,mt1,         ...,           mt14,mt15
 16          8         TExTE       mt0,mt2,mt4,mt6,mt8,mt10,mt12,mt14
 32          4         TExTE       mt0,     mt4,     mt8,      mt12
 64          8     (TE/2)x(TE/2)   mt0,mt2,mt4,mt6,mt8,mt10,mt12,mt14
```

For smaller TEW<32, the tile dimensions (TExTE) are kept the same as TEW=32. Every halving of TEW

halves the state required for a single tile and so the number of tiles increases by a factor of two. The storage of each TEW=32 tile is divided to form two TEW=16 tiles (e.g. `mt0` for TEW=32 forms `mt0` and `mt2` for TEW=16), and the storage of each TEW=16 tile is divided to form two TEW=8 tiles (e.g. `mt0` for TEW=16 forms `mt0` and `mt1` for TEW=8).

For TEW=64, the tile dimensions are reduced by a factor of 2 in each dimension (i.e. tiles are TE/2 x TE/2 elements), but the number of tiles increases to 8 as the tile size shrinks quadratically while the element width only doubles. The storage of each TEW=32 tile is divided to form two TEW=64 tiles (e.g. `mt0` for TEW=32 forms `mt0` and `mt2` for TEW=64).

The parameter ETE describes the tile dimensions as a function of TEW: `ETE = TEW < 64 ? TE : TE / 2`.

Additional ISA extensions describe the tile dimensions. The Zvt*N*t extension indicates `TE` ≥ `N`, e.g. Zvt32t indicates TE is at least 32.



The accumulator array dimensions (TE) may be determined dynamically with the `msettn` *instruction to enable writing TE-agnostic software.*

Tile Punning

The matrix tile state is reshaped for each supported TEW which results in tiles being combined to support wider accumulators. For example, 4 TEW=8 tiles (mt0..3) are combined to support a single TEW=32 tile (mt0).



Most software consistently accesses a tile using the same TEW, and so is indifferent to the interpretation of the tile under other TEWs. However, there are cases where software must access elements in a tile without knowing the TEW with which it was most recently written, including context switching and debugging. In particular, migrating contexts between harts with different tile-punning schemes is impractical. Hence, it is highly desirable to fix the tile-punning scheme.

Architecturally tagging the tiles with their TEWs has been proposed as an alternative to mandating a tile-punning scheme. Alas, the fact that tiles can be accessed in both rows and columns makes this scheme impractically expensive: every byte would need its own type tag.

We chose this particular layout because it enables efficient access to elements by both row and column, supporting consecutive accesses in either dimension without overly favoring one over the other.

The following figure illustrates how elements are combined and aliased across supported TEWs.

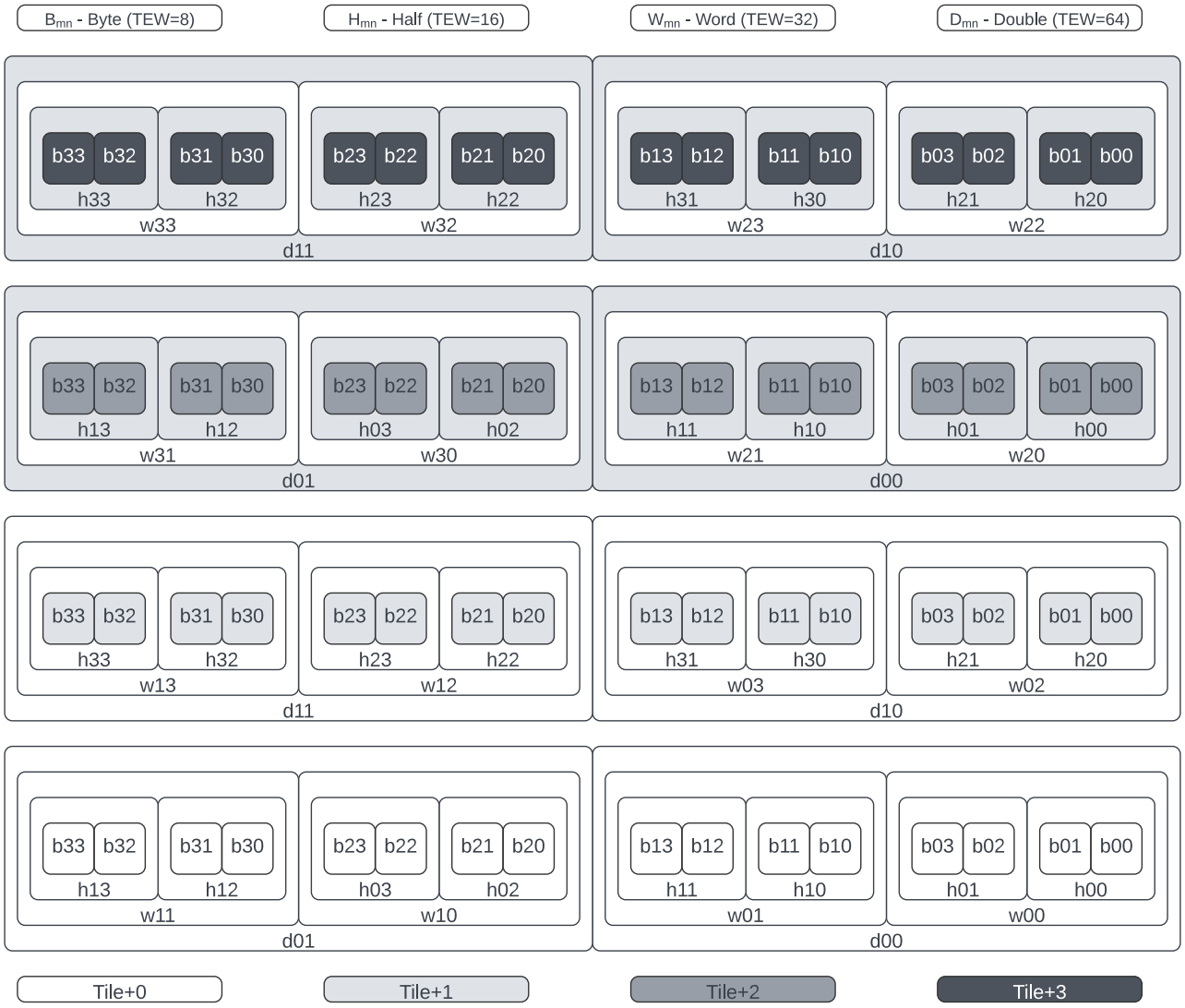


Figure 10. Tile Punning Aliasing

Viewing the tile state as a linearized buffer of 16 * TE * TE bytes, an element's position can be determined with the following algorithm.

```
unsigned punning (
  const unsigned TE,  // tile array dimension TExTE
  unsigned tile,      // addressed tile
  unsigned row,       // tile row index
  unsigned col,       // tile col index
  unsigned tew        // tile element width
) {
  unsigned ptile, minor_offset, major_offset;

  switch (tew) {
    case 8: {
      ptile = tile;
      minor_offset = (row % 4) * 4 + (col % 4);
      major_offset = (row / 4) * (TE / 4) + (col / 4);
      break;
    }
```

```
    case 16: {
      ptile = tile + ((row & 2) >> 1);
      minor_offset = (row % 2) * 4 + (col % 2) * 2 + ((col / 2) % 2) * 8;
      major_offset = (row / 4) * (TE / 4) + (col / 4);
      break;
    }
    case 32: {
      ptile = tile + 2 * (row / (TE / 2)) + ((col & 2) >> 1);
      minor_offset = (row % 2) * 8 + (col % 2) * 4;
      major_offset = ((row / 2) % (TE / 4)) * (TE / 4) + (col / 4);
      break;
    }
    case 64: {
      ptile = tile + (row / (TE / 4));
      minor_offset = (col % 2) * 8;
      major_offset = (row % (TE / 4)) * (TE / 4) + (col / 2);
      break;
    }
    default: return 0;
  }

  unsigned offset = (ptile * TE * TE) + (major_offset * 16) + minor_offset;

  return offset;
}
```

12.1.1.2. Matrix Type (`mtype`) CSR

The Zvtbase extension adds a new unprivileged read-only CSR, `mtype`, with address 0xC23. It contains the following fields:

```
bits      field
XLEN-1:24 reserved
    23:10 tm[13:0]
     9:8  reserved
     7:5  tk[2:0]
     4:2  reserved
     1:0  mtwiden[1:0]
```

`tm` can hold values from 0-TE, inclusive.

`tk` can hold values from 0-4, inclusive.

If `mtwiden`=0, the matrix unit is not configured.

12.1.1.3. Matrix-Multiply Operations

The general form of the matrix-multiply operations is:

```
C[tm,tn] += A[tk,tm]^T * B[tk,tn]
```

where the inputs A and B are each sourced from a vector register group, while the accumulator C is held in a matrix tile. A, B, and C are all specified using register specifier fields in the instruction encoding.

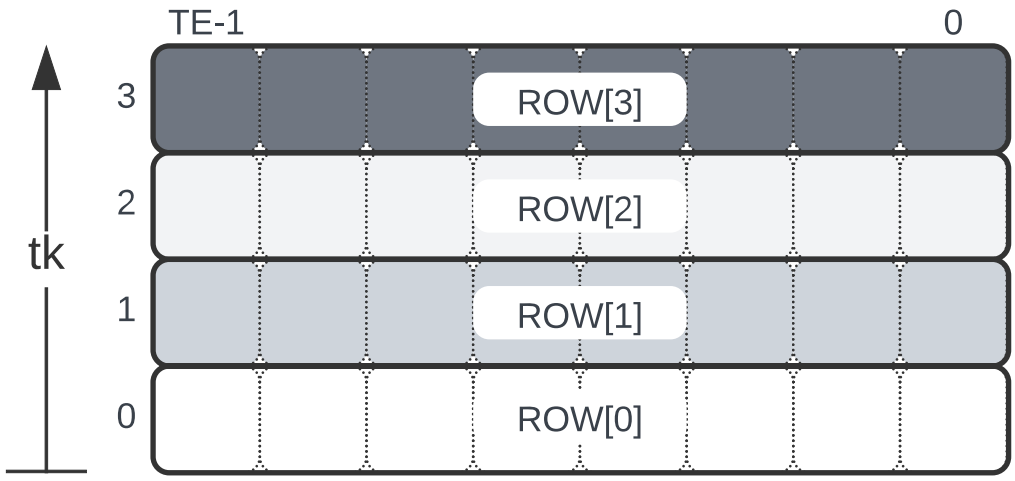


Figure 11. A or B Matrix Operand

The values of `tm`, `tn`, and `tk` are obtained from CSRs. `tm` and `tk` are fields in the `mtype` register. `tn` is derived from `vl`; it is equal to `min(vl, ETE)`. The new configuration instructions described below allow the values of `tm`, `tn`, and `tk` to be set in an implementation-agnostic manner from the application matrix dimensions.

Each vector register group input has `tk` rows, where each row is held separately in one or more vector registers, with the total number of vector registers constrained to fit in one eight-register group. Each row fits within a vector register group (i.e. an aligned power-of-2 number of vector registers) and the collection of rows also fits in a (potentially larger) vector register group. The value of `tk` sets the dimension of the dot products performed for each accumulator element and the maximum value of `tk` (KMAX) depends on the element widths, with a maximum value of 4.

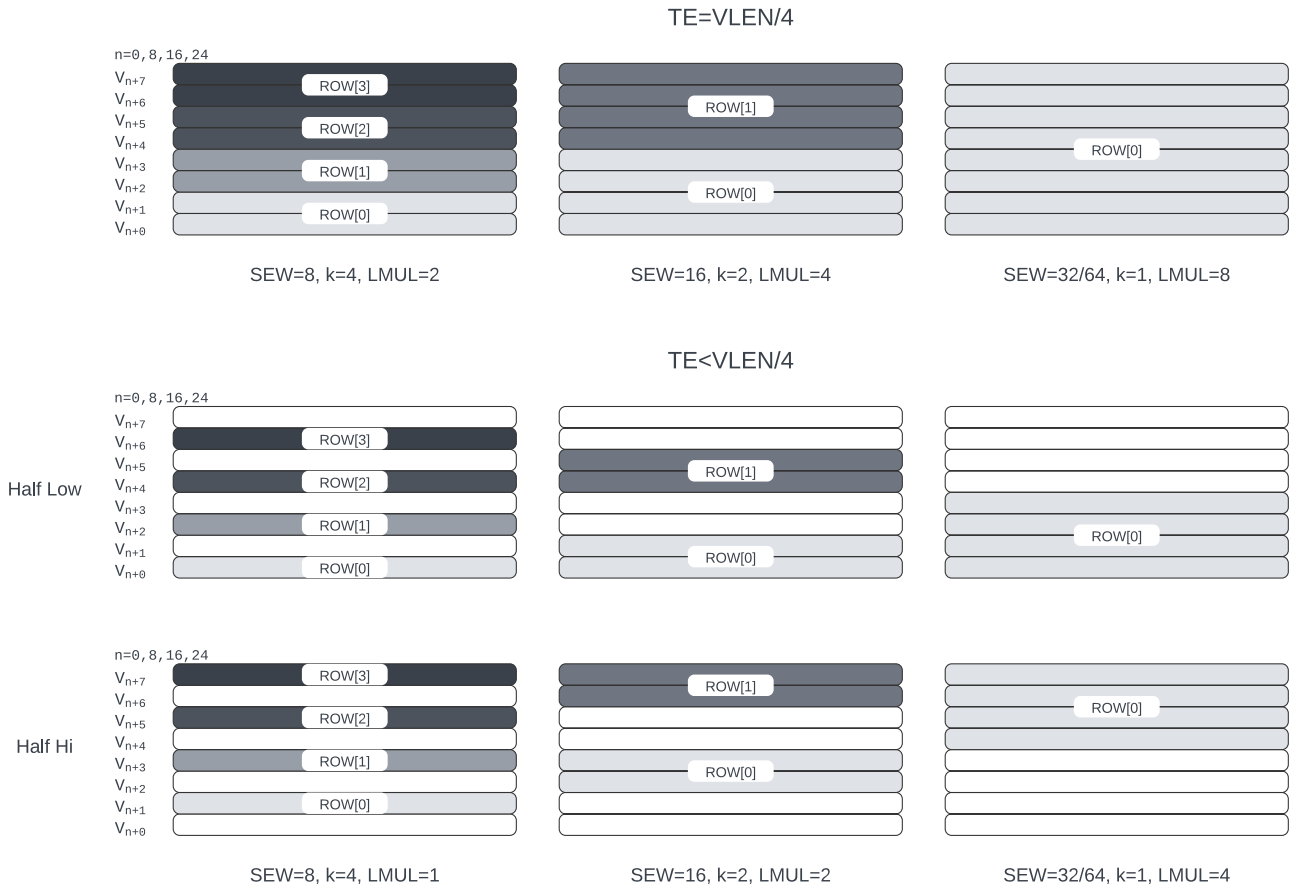


Figure 12. Matrix Operand Layout in Vector Register Group

This format allows existing vector unit-stride loads to be used to load each row of an input from a memory matrix held in row-major format.

Successive rows are separated by (8/KMAX) vector registers. For example, for vector register group specifier `v8`, and assuming KMAX=4 and LMUL=2, then `v8`..`v9` holds one matrix row, `v10`..`v11` holds the next row, `v12`..`v13` the next, and `v14`..`v15` the last. But when KMAX=4 and LMUL=1, then `v8`, `v10`, `v12`, and `v14` hold the successive rows of the matrix: the separation between rows is the same, but the odd-numbered vector registers are skipped.

Vector register specifiers must be divisible by LMUL. For example, when KMAX=4 and LMUL=2, `v8` is valid but `v9` is not. But when KMAX=4 and LMUL=1, both are valid: the former represents the even-numbered registers `v8`..`v14`, whereas the latter represents the odd-numbered registers `v9`..`v15`.



These definitions facilitate writing LMUL-agnostic software, but they do not preclude LMUL-conscious software from using all of the vector registers.

Vector register specifiers, when taken modulo 8, must be less than 8/KMAX. For example, when KMAX=4, `v10` is invalid.



This constraint prevents register groups from spanning EMUL=8 register-group boundaries, simplifying vector-unit implementations.

12.1.1.4. Matrix Configuration Instructions

Application matrices have varied sizes, and implementations will vary in the TE dimensions they support. The matrix configuration instructions allow a single binary matrix-multiply routine to work for any size application matrix multiply on any sized matrix-multiply implementation.



More optimized routines are possible when microarchitectural parameters are known to software.

As with the `vset` family of instructions, the matrix configuration instructions do not raise illegal-instruction exceptions as a consequence of `vtype`.vill holding the value 1.

The actual size of a matrix-multiply operation is determined by values in two CSRs: `tk` and `tm` are fields in the `mtype` register; `tn` is derived from the `vl` register. These CSRs are set using configuration instructions.

```
Encoding of matrix configuration instructions

31 29    28    27 25 24  23 22 20 19 17 16     15 14 12 11    7 6     0
 100 |   0    | 100 |  00  | 000 |       rs1     | 111 |  rd   |1010111| msettn
 100 |   0    | 100 |  00  | 001 |       rs1     | 111 |  rd   |1010111| msettm
 100 |   0    | 100 |  00  | 010 |       rs1     | 111 |  rd   |1010111| msettk
 100 | altfmt | 100 | vsew | 100 | 000 | mtwiden | 111 | 00000 |1010111|
msetmtypei
 100 |   0    | 010 |     rs2    |       rs1     | 111 | 00000 |1010111|
msetmtype
```

The existing `vset[i]vl[i]` instructions write zero to the `mtype` CSR.

The `msetmtype` instruction writes the `mtype` and `vtype` CSRs to the values in the `rs1` and `rs2` registers. First, `vl` is written with 0, and `mtype` is written with the value in `rs1`. Then, `vtype` is written with the value in `rs2`, as though by a `vsetvl` instruction. Finally, if the new `mtype.mtwiden` != 0, then `vma` and `vta` are set to 1, `vill` is set if the resulting TEW is unsupported, and `vlmul` is set as follows:

The `msetmtypei` instruction is defined similarly, except the new `mtype.mtwiden`, `vtype.vsew`, and `vtype.altfmt` fields are supplied by immediates.

```
msetmtype[i] behavior:

vl = 0
if msetmtypei:
  mtype = 0
  mtype.mtwiden = insn.mtwiden
  vtype = 0
  vtype.vsew = insn.vsew
  vtype.altfmt = insn.altfmt
else:
  mtype = rs1
  vtype = rs2

if vtype is invalid per vsetvl semantics:
  vtype.vill = 1

if mtype.mtwiden != 0:
  let TWIDEN = 1 << (mtype.mtwiden - 1)
  let TEW = SEW * TWIDEN
  let ETE = TEW < 64 ? TE : TE/2 // Effective number of elements along tile
```

```
edge
  let EVE = VLEN/SEW             // Effective number of elements in a vector
register

  vtype.vlmul = log2(min(8/KMAX, 8/TWIDEN, ceil(ETE/EVE)))
                         ^        ^           ^
                         |        |           +-matrix engine size constraint
                         |        +-matrix row/col must fit in vector register
group
                         +-leave space for K dimension

  vtype.vma = 1
  vtype.vta = 1

  vtype.vill |= (SEW * TWIDEN > ELEN)

  mtype.tk = min(mtype.tk, KMAX)
  mtype.tm = min(mtype.tm, LMUL*EVE, ETE)

if vtype.vill || mtype.mtwiden == 0:
  mtype = 0

if vtype.vill:
  vtype = 0
  vtype.vill = 1
```



The recommended assembly syntax for the `msetmtypei` *instruction is* `msetmtypei eM[alt], wN`*, where* `M` *gives SEW (8, 16, 32, 64),* `alt` *sets altfmt=1 when present, and* `N` *gives TWIDEN (0, 1, 2, 4), with* `mtype[4:2]` *implicitly 0 in this form.*

These instructions are included in the Zvtbase extension.

`msettn` instruction

This instruction sets `tn`/`vl` to the value in `rs1`, constrained by the current matrix configuration. If the matrix unit is not configured, i.e. `mtwiden`=0, the instruction writes `vl` as if by a `vsetvl` instruction using the current `vtype` setting. The new value of `tn`/`vl` is also written to the scalar destination register `rd`.

```
msettn behavior:

if mtype.mtwiden == 0:
  rd = vl = rs1 (following vsetvl semantics)
else:
  rd = vl = min(rs1, LMUL*EVE, ETE)
```



The `msetmtype` *and* `msettn` *instructions are intended to be used by context-switching routines. Because their semantics degenerate to those of* `vsetvl` *when the matrix unit is not configured, the usual* `vsetvl` *can be replaced with* `msetmtype` *and* `msettn`*; there is no need to use a dynamic branch to select between the two schemes.*

This instruction is included in the Zvtbase extension.

`msettm` and `msettk` instructions

These instructions set `tm` or `tk` to the value in `rs1`, constrained by the current matrix configuration. If the matrix unit has not been configured, i.e. `mtwiden`=0, the new `tm` or `tk` value is 0. The new value of `tm` or `tk` is also written to the scalar destination register `rd`.

```
msettm rd, rs1  // rd = tm = min(rs1, LMUL*EVE, ETE)
```

The `msettm` instruction sets `tm` to be the lesser of `rs1`, `LMUL`*`EVE`, and `ETE`.

```
msettk rd, rs1  // rd = tk = min(rs1, KMAX)
```

The `msettk` instruction sets `tk` to the lesser of `rs1` and KMAX.

The `msettm` and `msettk` instructions raise an illegal-instruction exception when `mtwiden`=0.

These instructions are included in the Zvtbase extension.



Unlike the `vsetvl` family of instructions, `msettn`, `msettm`, and `msettk` are not special-cased when `rd` and/or `rs1` is `x0`.

Supported SEW, TWIDEN, and KMAX values

The inner dimension specified by `tk` is used to support dot-product operations on narrower datatypes to increase efficiency. The ISA fixes maximum K values (KMAX) of matrix-multiply instructions according to SEW and TWIDEN to simplify ISA specification of these arithmetic operators.



Unlike other matrix accelerator architectures that support fusion of narrower datatypes by flattening K into both the M and N dimensions, this approach allows software to use standard matrix formats.

```
TEW=8

SEW  TWIDEN  KMAX
 8    1       4  (4-element dot product)
```

```
TEW=16

SEW  TWIDEN  KMAX
 8    2       4  (4-element dot product)
16    1       2  (2-element dot product)
```

```
TEW=32
```

```
SEW  TWIDEN  KMAX
 8    4       4  (4-element dot product)
16    2       2  (2-element dot product)
32    1       1
```

```
TEW=64

SEW  TWIDEN  KMAX
16    4       2  (2-element dot product)
32    2       1
64    1       1
```

12.1.1.5. Tile Subset Specifier (TSS)

For operations that move rows or columns between the matrix tiles and vector registers or memory, a scalar value is used to encode the set of tile elements that are accessed and the pattern by which they are accessed.

```
bits       meaning
XLEN-1:31  reserved
 30:27     tile specifier (0-15)
 26:24     pattern (0=row, 1=col, >1 reserved)
 23:0      index within pattern (e.g. row/col index)
```

When there are fewer than 16 tiles due to the specified TEW, the `log2(16 / NTiles)` LSBs of the tile-specifier field are ignored, such that this field always refers to a valid tile.

The 3-bit field for pattern currently only has two encodings defined, row and column. The pattern field defines how the index field is interpreted. For the currently defined row and column patterns, the index field is treated as an unsigned integer providing a one-dimensional index of the row or column. Row and column indices greater than or equal to ETE are reserved.



We expect most implementations will ignore reserved bits in TSS, taking `pattern` *mod 2, and taking the index within the pattern mod ETE.*

12.1.1.6. Load/Store Tile Subset to Memory

These instructions transfer matrix tile subsets between the tile state and memory. `vtle` and `vtse` support TEW values corresponding to all supported SEW settings (8..ELEN). Implementations must provide vector loads and stores with EEWs corresponding to all supported SEW settings (8..ELEN). These instructions are included in the Zvtbase extension.

```
// Load tile subset from memory, using TEW=EEW
vtle8  rs2, (rs1) // rs1 is memory address, rs2 is TSS
vtle16 rs2, (rs1) // rs1 is memory address, rs2 is TSS
vtle32 rs2, (rs1) // rs1 is memory address, rs2 is TSS
vtle64 rs2, (rs1) // rs1 is memory address, rs2 is TSS
```

```
// Store tile subset to memory, using TEW=EEW
vtse8  rs2, (rs1) // rs1 is memory address, rs2 is TSS
vtse16 rs2, (rs1) // rs1 is memory address, rs2 is TSS
vtse32 rs2, (rs1) // rs1 is memory address, rs2 is TSS
vtse64 rs2, (rs1) // rs1 is memory address, rs2 is TSS
```

```
31 29 28 27 26 25 24    20 19   15 14 12 11   7 6     0
  nf  mew  mop vm    rs2     rs1   width vd/vs3 opcode
e e e  1  0  0  1    rs2     rs1   1 1 1    0   0000111 // Load tile
e e e  1  0  0  1    rs2     rs1   1 1 1    0   0100111 // Store tile

eee
000  8
001 16
010 32
011 64
1xx reserved
```

The integer register specified by `rs1` holds the base address in memory. The integer register specified by `rs2` holds the tile subset specifier. These instructions do not support masks. EEW is statically encoded in the instruction.

These instructions always load and store the tile subset to a contiguous region of memory. For the currently defined row and column patterns, the elements of the row or column are stored in increasing index order in memory.

For these instructions, the body elements are those with indices in the range `[vstart, min(vl, ETE) - 1]`, and the tail elements are those with indices ≥ `min(vl, ETE)`.



Since `vl`=`tn` *holds the number of columns in the accumulator, i.e. the length of a row, these instructions are oriented towards loading and storing accumulator rows. When loading and storing accumulator columns, additional* `msettn` *instructions may be needed to program* `vl` =`tn` *to hold the number of rows, i.e. the length of a column, instead.*

For the purposes of exception handling, these instructions are considered to access memory in ascending order beginning at the byte determined by a function of the base address in `rs1`, the `vstart` value, and the instruction EEW. No memory is accessed below the elements specified by the `vstart` value.

The matrix extension follows the precise exception model of the vector extension, allowing memory to be updated past the element indicated by `vstart` on a trap.

Misaligned element addresses raise either a load/store address-misaligned exception or a load/store access-fault exception.



The vector extensions allow element addresses to be misaligned to facilitate automatic vectorization of existing scalar code that might use misaligned addresses. In contrast, software written for the Zvt extensions is new software, for which the standard ABIs require natural alignment. Disallowing misaligned accesses is therefore unlikely to cause incompatibilities, and doing so has the benefit of simplifying implementations and detecting programming errors.

If `mtwiden`=0, these instructions are reserved.

12.1.1.7. Move Tile Subset between Tile State and Vector Registers

Tile subset operations transfer elements between a matrix tile and a vector register group. These instructions support TEW values corresponding to all supported SEW settings (8..ELEN). These instructions are included in the Zvtbase extension.

```
// Move tile subset to vector register group, using TEW=SEW
vtmv.v.t vd, rs1  // vd is destination vector register, rs1 is TSS of source

// Move vector register group to tile subset, using TEW=SEW
vtmv.t.v rs1, vs2 // vs2 is source vector register, rs1 is TSS of destination
```

```
31          26 25 24    20 19   15 14 12  11   7 6     0
    funct6     vm    rs2     rs1   funct3 vd/vs3 opcode
    010000      1   11111    rs1   1 1 0    vd   1010111 // vtmv.v.t
    010111      1    vs2     rs1   1 1 0     0   1010111 // vtmv.t.v
```

For these instructions, the body elements are those with indices in the range [`vstart`, `min(vl, ETE) - 1`], and the tail elements are those with indices ≥ `min(vl, ETE)`.

If `mtwiden`=0, these instructions are reserved.

12.1.1.8. Matrix Arithmetic Instructions

These instructions all have the form C += A^T * B, where C is held in a matrix tile and A and B are supplied by vector register groups.

Unsupported `vtype` or `mtype` settings are reserved. An illegal-instruction exception is raised if `vstart` is nonzero.

```
# vtype holds SEW and altfmt; mtype holds TWIDEN
# vs2 specifies A vector register group
# vs1 specifies B vector register group
# mtd is C destination tile specifier

# datatype of vs2 is supplied by opcode, SEW
# datatype of vs1 is supplied by altfmt, SEW
# datatype of mtd is supplied by opcode, SEW, TWIDEN


# FP Matmuls (OCP, BF16, IEEE)

# SEW  TWIDEN  altfmt    vs2   vs1   mtd
#   8      4       0    E4M3  E4M3  FP32 (Zvtofp8fmm)
#   8      4       1    E4M3  E5M2  FP32 (Zvtofp8fmm)
#  16      2       0    FP16  FP16  FP32 (Zvtfp16fmm)
```

```
#  32      1       0    FP32  FP32  FP32 (Zvtfmm)
#  64      1       0    FP64  FP64  FP64 (Zvtdmm)
vtfmm.tvv     mtd, vs2, vs1

# SEW  TWIDEN  altfmt    vs2   vs1   mtd
#   8      4       0    E5M2  E4M3  FP32 (Zvtofp8fmm)
#   8      4       1    E5M2  E5M2  FP32 (Zvtofp8fmm)
#  16      2       1    BF16  BF16  FP32 (Zvtbf16fmm)
vtfmm.alt.tvv mtd, vs2, vs1


# Integer Matmuls

# SEW  TWIDEN  altfmt    vs2   vs1   mtd
#   8      4       0   uint8 uint8 int32 (Zvti8i32mm)
#   8      4       1   uint8  int8 int32 (Zvti8i32mm)
vtmmu.tvv mtd, vs2, vs1

# SEW  TWIDEN  altfmt   vs2   vs1    mtd
#   8      4       0   int8 uint8  int32 (Zvti8i32mm)
#   8      4       1   int8  int8  int32 (Zvti8i32mm)
vtmms.tvv mtd, vs2, vs1
```



There are numerous FP8 formats being developed and standardized, but the OCP MX formats appear to be gaining the most traction. Additional formats would ideally be supported with explicit conversions to BF16 or FP16 in the vector unit, as supporting all combinations natively would consume substantial opcode space. But native support for additional formats remains possible if sufficient demand arises.

These instructions use the values of `tm`, `tn`, and `tk` to determine what computations to perform. If any of these values are zero, then no computation is performed and no state is updated.

Any tile elements outside the range [0, tm-1] x [0, tn-1] are considered part of the tail and are handled using a tail-agnostic policy.

The invalid and overflow scalar floating-point exception flags are updated by the results of floating-point matrix operations. The inexact, underflow, and divide-by-zero flags are not updated, nor is the fixed-point saturation flag.

For SEW ≤ 16, floating-point products are bulk-normalized accumulated in fixed-point arithmetic, then converted to FP32, rounding to odd, according to the RVBNA Algorithm. Finally, they are added to the accumulator in IEEE 754 arithmetic, rounding to odd as described in RVBNA Rounding to Odd.

For SEW ≥ 32, floating-point products are computed in IEEE 754 arithmetic then rounded to a TEW-bit value, using the rounding mode specified by `frm`. They are then added to the accumulator in IEEE 754 arithmetic, using the same rounding mode.



Rounding the product before accumulating facilitates datapath sharing with the SEW ≤ 16 operations. It also reduces power, area, and back-to-back accumulation latency as compared to a fused multiply-adder, even if the FMA is cascaded to optimize this case. Future extensions can define fused multiply-accumulate instructions.

If the Zvtbf16fmm extension is implemented, the implementation must support selecting `altfmt`=1 when SEW=16. Similarly, if the Zvti8i32mm or Zvtofp8fmm extension is implemented, the implementation must support selecting `altfmt`=1 when SEW=8.

Matrix Arithmetic Instruction Encoding

The matrix arithmetic instructions are encoded in the second vector major opcode, OP-VE.

The `rd` field encodes the matrix tile number, represented with `t` in the diagram below. The tile-selection bits are allocated from the MSB of the `rd` field (e.g. instructions that do not support TEW=8 do not encode `t[0]`, and so use `rd[4:2]` to encode `t[3:1]`). Tile-selection bits not present in the instruction encoding are defined to be zero. If `t` encodes an invalid tile specifier, i.e. `t` modulo `(16 / NTiles)` is nonzero, the instruction is reserved.

```
31   26 25 24  20 19  15 14 12  1110 9 8 7 6     0
 funct6 vm   vs2    vs1  funct3     rd       opcode
 111100  1   vs2    vs1    001   t t t 0 0  1110111 // vtfmm.tvv
 111100  1   vs2    vs1    001   t t t 0 1  1110111 // vtfmm.alt.tvv

 111100  1   vs2    vs1    000   t t t 0 0  1110111 // vtmmu.tvv
 111100  1   vs2    vs1    000   t t t 0 1  1110111 // vtmms.tvv
```

12.1.1.9. Matrix Tile-Zero Instruction

The `vtzero` instruction writes 0 to each element of the `tm` by `tn` submatrix of the destination matrix tile. Any tile elements outside the range [0, tm-1] x [0, tn-1] are considered part of the tail and are handled using a tail-agnostic policy. `vtzero` supports TEW values corresponding to all supported SEW settings (8..ELEN). This instruction is included in the Zvtbase extension.

```
# Write zeros to tile
vtzero mtd

31   26 25 24  20 19  15 14 12  1110 9 8 7 6     0
 010000  1  11110  00000   110   t t t t 0  1010111 // vtzero
```

12.1.1.10. Context Status

The `mtype` CSR is considered part of the vector context; access to this CSR is controlled by the various `VS` fields.

As described below, additional `MS` context-status fields are added to reduce the cost of saving and restoring the matrix tile state. These fields do not affect accessibility of the `mtype` CSR.

Zvt Context Status in `mstatus`

A Zvt context status field, `MS`, is added to `mstatus[30:29]` and shadowed in `sstatus[30:29]`. It is defined analogously to the floating-point context status field, `FS`.

Attempts to execute any instruction that accesses the matrix tile state raises an illegal-instruction

exception when `mstatus`.MS is set to Off. Note, the `vset*` and `mset*` instructions do not access the tile state.

When `mstatus`.MS is set to Initial or Clean, executing any instruction that changes tile state will change `mstatus`.MS to Dirty. Implementations may also change `mstatus`.MS from Initial or Clean to Dirty at any time, even when there is no change in tile state.

If `mstatus`.MS is Dirty, `mstatus`.SD is 1; otherwise, `mstatus`.SD is set in accordance with existing specifications.

Zvt Context Status in `vsstatus`

When the hypervisor extension is present, a Zvt context status field, `MS`, is added to `vsstatus[30:29]`. It is defined analogously to the floating-point context status field, `FS`.

When V=1, both `vsstatus`.MS and `mstatus`.MS are in effect: attempts to execute any instruction that accesses matrix tile state raise an illegal-instruction exception when either field is set to Off.

When V=1 and neither `vsstatus`.MS nor `mstatus`.MS is set to Off, executing any instruction that changes tile state will change both `mstatus`.MS and `vsstatus`.MS to Dirty. Implementations may also change `mstatus`.MS or `vsstatus`.MS from Initial or Clean to Dirty at any time, even when there is no change in tile state.

If `vsstatus`.MS is Dirty, `vsstatus`.SD is 1; otherwise, `vsstatus`.SD is set in accordance with existing specifications.

If `mstatus`.MS is Dirty, `mstatus`.SD is 1; otherwise, `mstatus`.SD is set in accordance with existing specifications.

For the purposes of the `mstatus`.VS and `vsstatus`.VS fields, all Zvt instructions (including configuration instructions) are considered to modify vector state. For the purposes of the `mstatus`.FS and `vsstatus`.FS fields, the Zvt instructions that implicitly access `fcsr` are considered to modify floating-point state.

Zvt Context Discard Instruction

```
31   26 25 24  20 19  15 14 12  11   7 6     0
 funct6 vm   vs2    vs1  funct3   rd   opcode
 010000  1  11100  00000   110   00000 1010111 // vtdiscard
```

The `vtdiscard` instruction is provided to inform the runtime that the matrix tile state is no longer useful and need not be saved. `vtdiscard` raises an illegal-instruction exception if `mstatus`.MS=Off (or if V=1 and `vsstatus`.MS=Off). Otherwise, it changes `mstatus`.MS to Initial (and, if V=1, changes `vsstatus`.MS to Initial). This instruction is included in the Zvtbase extension.

`vtdiscard` raises an illegal-instruction exception if `vtype`.vill=1, but does not raise an exception as a result of `mtype.mtwiden` being zero.



`vtdiscard` *does not actually write the tile state. For ABI purposes, the tile state becomes UNSPECIFIED as a result of executing this instruction.*